Learn it from zero

A beginner's course in machine learning and NLP, taught entirely through the PrivyLens project you built.

> Send the report to [redacted] at [redacted].
>
> O O O O B-PERSON I-PERSON O B-EMAIL O

By the end you will understand that second line: how a computer learns to put a label on every word, why that needs a Transformer, and how each line of your project does it. No maths background needed.

Plan: three evenings. Evening 1 = lessons 1–4. Evening 2 = lessons 5–8. Evening 3 = lessons 9–11 and the quiz. Every lesson ends with a "Check yourself" box. If you can answer it out loud, move on.

The map of ideas

Everything in this guide hangs on one chain. Learn it as a story, not as a list.

```
Computers only understand numbers
  └→ so text must become numbers                    (lesson 3: tokens, embeddings)
       └→ a model needs to look at context          (lesson 5: attention)
            └→ many such layers = a Transformer     (lesson 6)
                 └→ it starts out knowing nothing   (lesson 2: random weights)
                      └→ we show it labelled examples and nudge it (lesson 7: training)
                           └→ we measure honestly   (lesson 8: precision, recall, F1)
```

> **The two big words**
>
> **ATML** (advanced topics in machine learning) in your project = the Transformer: attention, encoder, decoder, training, ablations. **NLP** (natural language processing) = handling text: tokenizer, BIO labels, span evaluation. Lessons 3–4 and 8 are mostly NLP. Lessons 2, 5–7, 9 are mostly ATML.

Lesson 1

What machine learning actually is

Normal programming: you write the rules. Machine learning: you show examples and the computer finds the rules.

Why rules fail for us

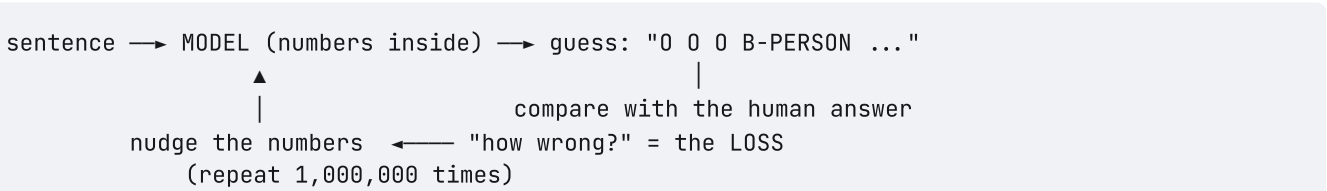
Try writing a rule for "this word is a person's name". "Rahul" is a name, "Rahul" in "Rahul Dravid Road" is a street, "Apple" is a company or a fruit. You can't list every name, and context changes meaning. But you can easily collect thousands of sentences where humans have marked the names. That is a **dataset**, and ML learns the pattern from it.

That is exactly what the regex baseline showed: for PERSON it scored **0.00**, because no pattern describes "a person".

The three ingredients

Ingredient	Meaning	In PrivyLens
Data	Examples with the right answers (labels)	~40,000 sentences where every private span is marked
Model	A function with adjustable numbers inside	The Transformer tagger, 1.56 million adjustable numbers
Training	Automatically adjusting those numbers until answers are mostly right	`train_tagger.py` on Colab

Supervised learning, in one picture



Our task is called **supervised** because a "supervisor" (the dataset labels) tells the model the right answer. The specific task, "give every word a label", is **sequence labelling**, also called **token classification** or **NER** (named entity recognition).

Check yourself

1) Why can't regex find names? 2) Name the three ingredients of ML. 3) Where do the labels in our dataset come from? (Answers: no pattern exists for "a person" · data, model, training · public dataset labels plus our synthetic generator, which records where it inserts each value.)

Lesson 2

Neural networks: the adjustable machine

A neuron is a weighted vote

A neuron takes numbers in, multiplies each by a **weight** (how much it matters), adds them up, and passes the result through a small bend called an **activation function** (ours is GELU). That is all.

```
inputs  x1 x2 x3        output = activation( w1·x1 + w2·x2 + w3·x3 + b )
weights w1 w2 w3                              ▲ the weights are what training changes
```

A **layer** is many neurons side by side. In PyTorch that is `nn.Linear(128, 256)`: it turns a list of 128 numbers into a list of 256 numbers using 128×256 weights. A **network** is layers stacked.

Parameters = all the weights

Your tagger has 1,556,887 parameters. At the start they are random, so the model outputs nonsense. **Training is just finding good values for all 1.5 million numbers.**

How the numbers get adjusted

1. **Forward pass**: feed a sentence in, get a guess out.
2. **Loss**: one number saying how wrong the guess was. We use cross-entropy, which punishes confident wrong answers the hardest.
3. **Backward pass** (backpropagation): the computer works out, for every weight, "if I increased you slightly, would the loss go up or down?" This is called the gradient. PyTorch does it with `loss.backward()`.
4. **Update**: move every weight a tiny step in the direction that lowers the loss. `opt.step()`. The step size is the **learning rate** (5e-4 = 0.0005).

Picture a hiker in fog trying to reach the valley floor: feel the slope under your feet, step downhill, repeat. That is **gradient descent**. AdamW is a smarter hiker that adapts its step per weight.

> **Where in the code**
> In `train_tagger.py`, these lines are the whole story: `loss = F.cross_entropy(...)`, `opt.zero_grad()`; `loss.backward()`, `opt.step()`. Everything else is plumbing.

Check yourself

What does "the model has 1.56M parameters" mean? What is the loss? What does the learning rate control? (Answers: 1.56 million adjustable numbers · a score of how wrong the guess is · how big each adjustment step is.)

Lesson 3 · NLP

Turning text into numbers

A model can only do arithmetic. So text goes through two conversions.

```
"Rahul Sharma"  ──tokenizer──►  [3052, 811, 4020]  ──embedding──►  3 lists of 128 numbers each
  (text)                          (token ids)                          (vectors the model can us
```

Step 1: Tokenization

Cut text into pieces (**tokens**) and give each piece an id number. Three options:

Cut by	Problem
Whole words	A new name or ID the model never saw becomes "unknown". The vocabulary also becomes huge.
Single characters	Nothing is unknown, but sentences become very long and each letter means little.
Pieces (BPE)	Common words are one piece, rare strings split into smaller known pieces. Nothing is unknown. This is what we built.

Worked example: how BPE learns its pieces

Tiny training text: **low** ×5, **lower** ×2, **newest** ×6. Start with every word as separate letters, then repeatedly merge the most frequent neighbouring pair.

Round	Most frequent pair	Count	New token	Words now
Start	–	–	–	l o w · l o w e r · n e w e s t
1	w + e	8 (2 in lower, 6 in newest)	`we`	l o w · l o w e r · n e we s t
2	l + o	7 (5 + 2)	`lo`	lo w · lo we r · n e we s t
…	repeat until the vocabulary has 8,000 tokens			

Our `bpe.py` does exactly this. It keeps the list of merges in order; to tokenize new text it applies the earliest-learned merges first. A name it has never seen simply ends up as smaller pieces. Special rule from `pretokenize.py`: `@ . -` stay inside a word, so `rahul@example.com` is not shredded.

Key detail: each token also remembers its character position. "Sharma" = characters 6–12. That is how a label on a token turns back into a black bar over exact characters.

Step 2: Embeddings

A token id is just an index. The model needs meaning, so each id is looked up in a table (`nn.Embedding`, 8,000 rows × 128 columns) and becomes a vector of 128 numbers. At first the rows are random. Training moves them so that tokens that behave alike (two first names) end up with similar numbers. That table is 1,024,000 of our 1.56M parameters, two thirds of the model.

Check yourself

Why BPE rather than whole words? Why does each token store character offsets? What is an embedding? (Answers: never unknown, small vocabulary · so predictions map back to exact characters · a learned list of 128 numbers for each token.)

Lesson 4 · NLP

Sequence labelling and BIO tags

Our model does not rewrite the sentence. It puts a sticker on every token. The sticker set is the **BIO scheme**:

Tag	Meaning
`O`	Outside: not private
`B-PERSON`	Beginning of a person's name
`I-PERSON`	Inside (continuing) the same name

```
Send  the  report  to   Rahul     Sharma    at   rahul@example.com   .
 O     O     O      O   B-PERSON  I-PERSON  O    B-EMAIL             O
```

Why B and I? Without them, "Rahul Sharma met Amit Shah" would read as one four-word name. B marks where a new name starts.

We have 11 entity types, each with a B and an I, plus O: 2×11 + 1 = **23 tags** (`schema.py`). The model's last layer outputs 23 scores for every token; the highest wins.

Spans: back to characters

A **span** = (start, end, label) in characters. Our data stores only spans (the single source of truth). Two functions in `align.py` convert in both directions:

- `to_bio` : spans + token positions → tags, used to build **training answers**.
- `to_spans` : tags + token positions → spans, used to turn **predictions** into redactions.

Check yourself

Label by hand: "Priya works at Infosys." (Answer: B-PERSON O O B-ORGANIZATION O. The full stop is its own token.) Why 23 tags? (11 types × 2 + O.)

Lesson 5 · ATML

Attention: the one idea that matters

To label a word you often need the words around it. Attention lets each word look at every other word and borrow what is useful.

The problem

Is "Sharma" a name? On its own, unclear. But next to "Rahul", after "Send the report to", it clearly is. The model needs **context**. Older models read left to right and forgot. Attention reads everything at once.

The library analogy (Query, Key, Value)

Every word makes three versions of itself, using three small weight tables the model learns:

Name	Think of it as
Query	"What am I looking for?" (written on a sticky note)
Key	"What do I contain?" (the label on each book's spine)
Value	"The actual content I can give you" (the book itself)

Each word compares its query with every word's key. A good match = high score = "pay attention to this word". Then it collects a weighted mix of the values.

A worked example with real numbers

Suppose, when processing "Sharma", the match scores against the three words *Rahul, report, to* come out as 2.0, 1.0 and 0.1.

1. **Softmax** turns scores into percentages that add to 100%: $e^{2.0}$=7.39, $e^{1.0}$=2.72, $e^{0.1}$=1.11, total 11.21. Weights: **66%, 24%, 10%**.
2. New "Sharma" vector = 0.66 × value(Rahul) + 0.24 × value(report) + 0.10 × value(to).
3. So "Sharma" now carries a lot of "I am next to Rahul" information, which makes B-PERSON / I-PERSON an easy call.

The real formula, in our code

```
scores = q @ k.transpose(-2, -1) / math.sqrt(self.dk)     # how much each word cares about each o
scores = scores.masked_fill(~mask, very_negative)         # padding gets ~0 weight after softmax
w = scores.softmax(-1)                                    # percentages
out = w @ v                                               # weighted mix of values
```

Written maths: $\mathbf{Attention(Q,K,V) = softmax(Q \cdot K^T / \sqrt{d_k}) \cdot V}$. The division by $\sqrt{d_k}$ only stops scores from growing too large (which would make softmax too extreme to learn from).

Multi-head attention

Do this 4 times in parallel, each "head" with its own Q/K/V tables, each free to watch for a different clue (the word before, a title like "Mr.", an @ sign). Then glue the 4 results together. With 128 numbers and 4 heads, each head works with 32. Our ablation: 4 heads beat 2 and 8.

Position: attention can't tell order by itself

Attention treats the sentence like a bag of words. We add a position signal (sine and cosine waves at different speeds) to each vector, so the model can tell "Rahul Sharma" from "Sharma Rahul". Fixed waves generalised better than a learned position table in our ablation.

Masks

Padding mask: sentences in a batch are padded to equal length; masking stops attention looking at padding.
Causal mask: used only in the decoder, forbids looking at later words (lesson 9).

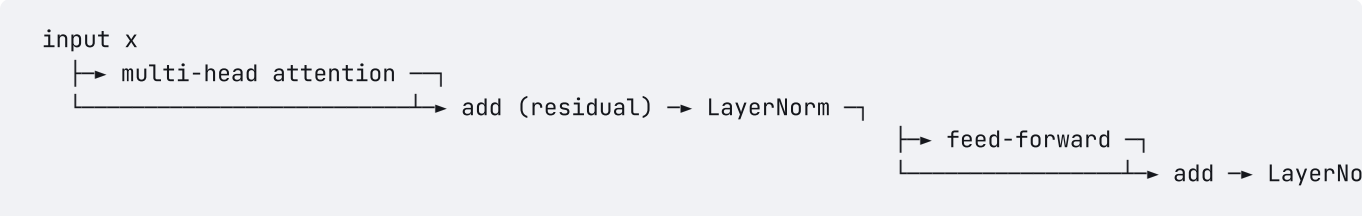
Check yourself

Explain Q, K, V with the library analogy. What does softmax do? Why do we need positional encoding? (Answers: Q = what I seek, K = what each word offers, V = the content · turns scores into percentages · attention alone ignores order.)

Lesson 6 · ATML

Building the Transformer from the pieces

One encoder block



Part	Why it is there
Attention	Mix in information from other words.
Residual ("add")	Keep the original information, output = input + change. Makes deep stacks trainable because the signal has a short path through.
LayerNorm	Rescales the numbers so they stay in a healthy range and training is stable.
Feed-forward	A small two-layer network applied to each word on its own (128→256→128) to "think about" what attention gathered.

Why stack several blocks?

Layer 1 might learn "this word follows 'to'". Layer 2 can combine that with other findings. Depth = more reasoning steps. Ablation proof: 2 → 4 layers gave +5.5 F1.

The whole tagger, with shapes

B = sentences per batch, T = tokens per sentence.

Step	Shape out	Code
token ids	B × T	`ids`
embedding (× √128)	B × T × 128	`self.emb(ids)`
+ positions, dropout	B × T × 128	`self.drop(self.pos(...))`
4 encoder blocks	B × T × 128	`for blk in self.blocks: x = blk(x, mask)`
linear head	B × T × 23	`self.head(x)`

Output: 23 scores per token. Pick the biggest, and you have the BIO tag. **That is the entire main model**: 61 lines in `encoder_tagger.py` on top of 127 lines in `layers.py`.

Dropout (0.1) randomly switches off 10% of numbers during training so the model can't lean on any single path. It helps against overfitting.

Check yourself

Draw one encoder block. Why a residual connection? What is the output shape of the tagger? (B × T × 23.)

Lesson 7 · ATML

Training: how the model actually learns

The vocabulary of a training run

Word	Meaning, with our numbers
Batch	32 sentences processed together (faster, steadier).
Epoch	One full pass over all ~40k training rows. We ran 25.
Step	One batch = one weight update.
Learning rate	Step size, 5e-4. Too big: jumps over the valley. Too small: takes forever.
Warm-up + cosine decay	Start gently for the first 5% of steps, peak, then shrink smoothly toward zero as the model settles.
Gradient clipping	Cap huge updates at 1.0 so one odd batch can't wreck the model.
Weight decay	A small pull towards smaller weights; reduces overfitting.

Three datasets, three jobs

```
TRAIN (39,806 rows)  the model learns from these
VAL   (3,956 rows)   after each epoch we score here and keep the best checkpoint
TEST  (3,993 rows)   final exam, never used for decisions
+ UNSEEN TEMPLATES (595) and CAPS (1,997): harder exams
```

Why not just test on training data? Because a model can **memorise**. Like a student who memorised last year's paper: perfect on it, lost on new questions. Testing on data the model never saw shows real ability.

Overfitting, in our own run

Training loss kept falling but validation F1 flattened at ~0.87 from epoch 20: the model was starting to memorise. We guard with dropout, weight decay, and by keeping only the best-validation checkpoint.

The overfit sanity check (clever, remember it)

Before the real run, train on only 10 sentences for 150 epochs with no dropout. A correctly wired model must memorise them, loss near 0. If not, there's a bug. This catches mistakes in minutes, not after hours of Colab.

A real bug story (great viva material)

The first full run scored F1 0.21. Cause: PyTorch's default embedding start values were ~11× larger than the position signal, so word identity drowned out position and attention saturated. Fix: initialise with small values (std $d^{-0.5}$). Lesson: initialisation matters, and tests plus sanity checks find such bugs.

Why Colab?

Your laptop CPU can run the code but training is slow. A Colab GPU does thousands of multiplications in parallel. We wrote and tested on CPU (tiny runs), then trained on GPU. Data is rebuilt there by scripts, because

the dataset licence forbids redistribution.

Check yourself

Why three splits? What is an epoch? Why try to overfit 10 examples on purpose? (Answers: learn, choose, final exam · one pass over training data · proves the code can learn at all.)

Lesson 8 · NLP

Evaluation: measuring honestly

Why plain accuracy is useless here

Most tokens are `O` (not private). A model that outputs "O" everywhere is ~95% accurate and hides nothing. So we score **spans**, not tokens.

Precision, recall, F1 with a worked example

The text really contains 10 private spans. The model hides 9 spans, of which 8 are exactly right.

Metric	Question it answers	Calculation
Precision	Of what we hid, how much was really private?	8 / 9 = 0.889
Recall	Of what was private, how much did we hide?	8 / 10 = 0.800
F1	**One number balancing both (harmonic mean)**	**2·P·R / (P+R) = 0.842**

A span only counts as correct if start, end **and** label all match exactly (`metrics.py`). Hiding half a name counts as wrong.

For privacy, recall is the one that matters

- Low **precision** = we hid harmless words too (annoying: "Mail me" became [PERSON_1] me).
- Low **recall** = a private value stayed visible (the failure the product exists to prevent).

So we added three privacy-specific metrics:

Metric	Meaning
Leaked spans	% of private spans not fully covered. Ignores the label: hiding an email as [PHONE] still hides it.
Leaked characters	% of private characters left visible (catches half-hidden names).
Over-redaction	% of harmless characters wrongly hidden (usability).

Reading the headline numbers

Regex: precision 0.72 but recall 0.23: when it fires it is usually right, but it misses most things. The production hybrid: F1 0.933, 2.8% leaked.

Check yourself

A model hides 20 spans, 15 correct, and the text had 30 private spans. Compute P, R, F1. (P = 0.75, R = 0.50, F1 = 0.60.) Which is worse for privacy, low P or low R? (Low R.)

Lesson 9 · ATML

The encoder-decoder, and why it lost

Two kinds of tasks

Task	Tool	Example
Label each input word	**Encoder only**	our tagger
Write new text	**Encoder + decoder**	translation, summaries, our seq2seq experiment

How the decoder works

The **encoder** reads the whole input into rich vectors. The **decoder** then writes the output **one token at a time**, each new token chosen using (1) what it has written so far and (2) the encoder's reading of the input.

```
input:  Send the report to Rahul Sharma at rahul@example.com.
output: Send the report to [PERSON] at [EMAIL].     (written token by token)
```

A decoder block has three parts instead of two:

1. **Masked self-attention**: look at the words written so far, never the future (causal mask: a triangle of allowed positions).
2. **Cross-attention**: queries come from the decoder, keys and values from the encoder. This is "look back at the input".
3. **Feed-forward**, each step with residual + LayerNorm.

Same `MultiHeadAttention` class in the code; passing `kv=encoder_output` turns it into cross-attention.

Teacher forcing and greedy decoding

- **Teacher forcing** (training): at every step feed the decoder the *correct* previous word, not its own guess. Fast, stable learning.
- **Greedy decoding** (using it): at every step pick the single most likely next token. If it errs once, every later token builds on the mistake.

Why it lost (the project's key finding)

It must **reproduce every unchanged word perfectly** and swap only private ones. Small models don't. On the test set only 23% of outputs were exactly right, ordinary words changed in 35% of rows, and it duplicated, corrupted or invented tokens. A tagger only labels, so it **cannot** damage the text. Right tool for the job.

Check yourself

Why can't the decoder see future words during training? What is cross-attention? Why is tagging safer than generating? (Answers: it would cheat by copying the answer · decoder looking at the encoder's output · labels cannot alter text.)

Lesson 10 · ATML

Pretrained models, DistilBERT, ONNX and ablations

Pretraining and fine-tuning

Our model started knowing nothing, and had to learn English from ~40k sentences. **DistilBERT** was first trained by others on enormous amounts of text, so it already understands language; we only **fine-tuned** it (4 epochs) to our labels. Like hiring a graduate versus teaching a child to read.

	Ours (from scratch)	DistilBERT (fine-tuned)
Parameters	1.56M	65.2M (40× more)
Size	5.5 MB (ONNX)	~261 MB
F1 on test	0.921 (hybrid 0.933)	0.956
Used in product?	Yes	No, reference only

Honest summary: it is better; ours is far smaller and fully ours, and the hybrid closes much of the gap.

ONNX

A universal model file. After training, `export_onnx.py` converts `tagger.pt` (PyTorch only) into `tagger.onnx`, runnable without PyTorch, even in a browser, and verifies the outputs match (difference ~1e-5).

Ablation = controlled experiment

Change **one** thing, keep everything else equal, compare. Our results:

- Layers 2 → 4: big gain. 4 → 6: small gain. **Diminishing returns**, so we picked 4.
- Heads 2 / 4 / 8: 4 best.
- Sinusoidal vs learned positions: similar on test, learned much worse on unseen templates.
- Vocabulary 4k / 8k / 16k: 16k worse because rare tokens get too little training.

Caveat to say yourself: one random seed each, so trust the trends, not the decimals.

Hybrid engine

Combine rules and learning. Regex is perfect on rigid formats (PAN, URL). The model understands meaning (names, places). Where they overlap, a priority rule decides (precise regex > model > loose regex). Our first rule scored worse than the model alone; we measured, found loose regex was overriding correct model labels, and fixed it. That is science working properly.

Check yourself

What is fine-tuning? Why isn't DistilBERT in the product? What does an ablation prove? (Answers: extra training of a pretrained model on our task · too big, and the project is about building our own · which design choice actually matters.)

Lesson 11

Read your own code: a guided tour

Open each file beside this page. Each starts with a docstring that explains it in plain English.

#	File	What to look for
1	`privylens/schema.py` (11 lines)	The 11 entities and the 23 tags. Shortest file, the vocabulary of the project.
2	`data/synthetic.py`	Templates like "Send it to {PERSON}"; each fill records its character span.
3	`tokenizer/pretokenize.py`, `bpe.py`	`train` = the merge loop from lesson 3. `encode` = returns ids **and** offsets.
4	`tokenizer/align.py`	`to_bio` and `to_spans` from lesson 4.
5	`models/layers.py`	Read `MultiHeadAttention.forward` line by line against lesson 5, then `EncoderBlock`.
6	`models/encoder_tagger.py`	The 5-step table from lesson 6, then `predict()` (windows of 128 tokens, averaging overlaps).
7	`training/train_tagger.py`	The loop: forward, loss, backward, step, then score on val and save the best.
8	`evaluation/metrics.py`	Precision, recall, F1 and the three leakage numbers.
9	`baselines/regex_detector.py`	The patterns, 31 lines.
10	`engine/pipeline.py`	`snap`, `merge`, `redact`, then class `Engine.sanitise`, the full story in ~100 lines.
11	`api/server.py`, `extension/content.js`	How the browser calls the engine.

> **How to read code you don't know**
> Don't understand every line. Ask of each function only: what goes in, what comes out, and which lesson explains it. Then you can answer "where did you implement attention?" with the file and function name.

Commands to see it run

```
python -m pytest tests                          # all tests: proves the pieces behave
python -m privylens.schema                      # prints "23 tags OK"
python -m privylens.tokenizer.bpe 8000 x.json   # (re)trains a tokenizer, shows tokens for a samp
python -m privylens.engine.pipeline             # scores the hybrid engine on the three test sets
```

Self-test

Cover the answers at the bottom. Answer aloud in one or two sentences. Redo what you miss tomorrow.

1. What is the difference between normal programming and machine learning?
2. What is a parameter? How many does our tagger have, and where are most of them?
3. List the four steps of one training update.
4. Why does BPE never meet an unknown word?
5. Label: "Meera lives in Pune." with BIO tags.
6. Explain Query, Key and Value with an analogy.
7. Why do we divide by $\sqrt{d_k}$, and what does softmax do?
8. Why add positional encoding?
9. What does the residual connection do? What does LayerNorm do?
10. Why are there train, validation and test sets?
11. What is overfitting and how did we see it?
12. A model hides 12 spans, 9 correct, 15 private spans exist. P, R, F1?
13. Why is recall the key privacy metric?
14. What is teacher forcing?
15. Why did the encoder-decoder lose to the tagger?
16. What is fine-tuning? Why is DistilBERT only a reference?
17. Why does regex still matter if you have a neural model?
18. What did the unseen-templates test prove?
19. How was the ALL-CAPS name bug fixed?
20. What is ONNX for?

Answers

1. Programming: you write the rules. ML: you give examples and the program finds the rules.
2. An adjustable number (weight). 1.56M; about 1.02M are in the embedding table.
3. Forward pass, compute loss, backward pass (gradients), update weights.
4. It can break any string down to single characters, which are always known.
5. B-PERSON O O B-LOCATION O (the full stop is its own token).
6. Library: Query is what I'm looking for, Key is each book's spine label, Value is the book's content. Match query to keys, then read the best-matching books.
7. Keeps scores moderate so softmax isn't extreme. Softmax turns scores into percentages summing to 100%.
8. Attention ignores word order; the sin/cos signal injects it.
9. Residual: output = input + change, so nothing is lost and deep stacks train. LayerNorm: keeps numbers well-scaled and training stable.
10. Train to learn, validation to pick the best checkpoint, test as an untouched final exam.
11. Memorising training data. Training loss kept falling while validation F1 plateaued near 0.87 after epoch ~20.
12. P = 9/12 = 0.75, R = 9/15 = 0.60, F1 = 2×0.75×0.60 / 1.35 = 0.667.

13. A missed value is a leak, the exact failure the product exists to prevent; a false alarm is only an inconvenience.
14. While training the decoder, feed it the true previous token rather than its own guess.
15. It must regenerate all the ordinary words perfectly, and it didn't: it changed text in 35% of rows. A tagger only labels.
16. Taking a pretrained model and training it a little on our task. It is 40× bigger (261 MB), so it is the upper-bound comparison, not the product.
17. It is perfect on rigid formats (email, URL, PAN) where the neural model is weaker; the two are complementary.
18. That the model generalises to new sentence shapes, not only memorised templates: tagger F1 0.787 and hybrid 0.879 there.
19. Only 41 of 17,772 PERSON examples were capitalised, so we added upper/lower-case copies to training, retrained the tokenizer and model, and added word snapping. Left-visible fell from 40.1% to 6.8%.
20. A portable model file that runs without PyTorch, even in a browser (5.5 MB).

Glossary at a glance

Word	One line
Weight / parameter	An adjustable number inside the model.
Loss	A score of how wrong the model is. Training lowers it.
Gradient	Which direction to nudge a weight to lower the loss.
Token	A piece of text the model reads.
Embedding	A learned list of numbers per token.
Attention	Every word looks at every other and takes what's useful.
Head	One of several parallel attentions, each with its own focus.
Encoder / Decoder	Reads the input / writes the output token by token.
BIO	Begin / Inside / Outside labels for spans.
Epoch / batch	One pass over the data / a group of 32 examples.
Overfitting	Memorising training data, failing on new data.
Precision / recall / F1	Of what we hid, how much was right / of what was private, how much we hid / the balance of both.
Fine-tuning	Extra training of a pretrained model.
Ablation	A one-change-at-a-time experiment.

Next step
Once you can do the self-test, open
PrivyLens_Viva_Guide.pdf
and read its questions section. This guide teaches the ideas; that one drills the answers.